

This guide focuses on SQL questions commonly asked in Data Engineering interviews.

Unlike traditional SQL interview preparation that focuses on syntax and definitions, this guide emphasizes real-world problem-solving, data transformation, deduplication, aggregations, reporting logic, and production-oriented scenarios.

The objective is to develop SQL thinking rather than memorizing queries.

Dataset Setup

Orders Table

```
CREATE TABLE orders (
```

```
    order_id INT,
```

```
    customer_id INT,
```

```
    order_date DATE,
```

```
    amount DECIMAL(10,2),
```

```
    status VARCHAR(20)
```

```
);
```

```
INSERT INTO orders VALUES
```

```
(1, 101, '2024-01-01', 500, 'completed'),
```

```
(2, 101, '2024-01-05', 300, 'completed'),
```

```
(3, 102, '2024-01-06', 700, 'cancelled'),
```

```
(4, 103, '2024-01-07', 1000, 'completed'),
```

```
(5, 101, '2024-02-01', 900, 'completed'),
```

```
(6, 102, '2024-02-03', 400, 'completed'),
```

```
(7, 104, '2024-02-05', 800, 'completed');
```

Customer Events Table

```
CREATE TABLE customer_events (  
    customer_id INT,  
    event_type VARCHAR(50),  
    event_time DATETIME  
);
```

```
INSERT INTO customer_events VALUES  
  
(101, 'login', '2024-01-01 10:00:00'),  
(101, 'purchase', '2024-01-01 10:20:00'),  
(101, 'logout', '2024-01-01 10:40:00'),  
(102, 'login', '2024-01-02 11:00:00'),  
(102, 'logout', '2024-01-02 11:30:00'),  
(103, 'login', '2024-01-03 09:00:00'),  
(103, 'purchase', '2024-01-03 09:15:00');
```

Employees Table

```
CREATE TABLE employees (  
    emp_id INT,  
    emp_name VARCHAR(50),  
    dept_id INT,
```

```
    salary INT
);

INSERT INTO employees VALUES
(1, 'Amit', 10, 50000),
(2, 'Riya', 10, 70000),
(3, 'Neha', 10, 70000),
(4, 'Rahul', 20, 60000),
(5, 'Karan', 20, 90000),
(6, 'Priya', 30, 80000);
```

SQL Problem Solving Questions

Question 1

Find duplicate customer IDs.

Answer

```
SELECT
    customer_id,
    COUNT(*) AS record_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

What Interviewers Check

- GROUP BY
 - HAVING
 - Duplicate detection
 - Data quality understanding
-

Question 2

Find the latest order for each customer.

Answer

```
SELECT *
FROM (
    SELECT *,
        ROW_NUMBER() OVER(
            PARTITION BY customer_id
            ORDER BY order_date DESC
        ) rn
    FROM orders
) x
WHERE rn = 1;
```

What Interviewers Check

- Window Functions
 - Deduplication Logic
 - Latest Snapshot Pattern
-

Question 3

Find second highest salary department-wise.

Answer

```
SELECT *  
FROM (  
    SELECT *,  
        DENSE_RANK() OVER(  
            PARTITION BY dept_id  
            ORDER BY salary DESC  
        ) salary_rank  
    FROM employees  
 ) x  
WHERE salary_rank = 2;
```

What Interviewers Check

- DENSE_RANK vs ROW_NUMBER
 - Tie Handling
 - Window Functions
-

Question 4

Calculate running revenue.

Answer

```
SELECT  
    order_date,
```

```
amount,  
SUM(amount) OVER(  
    ORDER BY order_date  
    ) running_revenue  
FROM orders  
WHERE status='completed';
```

What Interviewers Check

- Window Aggregations
 - Reporting Use Cases
-

Question 5

Find customers who logged in and later made a purchase.

Answer

```
SELECT DISTINCT l.customer_id  
FROM customer_events l  
JOIN customer_events p  
ON l.customer_id = p.customer_id  
WHERE l.event_type='login'  
AND p.event_type='purchase'  
AND p.event_time > l.event_time;
```

What Interviewers Check

- Self Joins

- Event Analytics
 - User Journey Analysis
-

Question 6

Find customers who never completed any order.

Answer

```
SELECT DISTINCT o.customer_id
FROM orders o
WHERE NOT EXISTS (
    SELECT 1
    FROM orders c
    WHERE c.customer_id=o.customer_id
    AND c.status='completed'
);
```

What Interviewers Check

- Anti Join Pattern
 - NOT EXISTS
 - NULL Handling
-

Question 7

Find monthly revenue.

Answer

```
SELECT
```

```
DATE_FORMAT(order_date,'%Y-%m') AS month,  
SUM(amount) AS revenue  
FROM orders  
WHERE status='completed'  
GROUP BY DATE_FORMAT(order_date,'%Y-%m');
```

What Interviewers Check

- Aggregation
 - Business Reporting
 - Time-Based Analysis
-

Question 8

Find customers whose revenue increased month-over-month.

Answer

```
WITH monthly_sales AS (  
SELECT  
customer_id,  
DATE_FORMAT(order_date,'%Y-%m') AS month,  
SUM(amount) revenue  
FROM orders  
GROUP BY customer_id,  
DATE_FORMAT(order_date,'%Y-%m')  
)
```



```
SELECT *
FROM (
    SELECT *,
        LAG(revenue)
        OVER(
            PARTITION BY customer_id
            ORDER BY month
        ) prev_revenue
    FROM monthly_sales
) x
WHERE revenue > prev_revenue;
```

What Interviewers Check

- CTEs
 - LAG Function
 - Trend Analysis
-

Question 9

Deduplicate records and keep latest version.

Answer

```
WITH ranked_data AS (
    SELECT *,
        ROW_NUMBER() OVER(
            PARTITION BY order_id
```

```
ORDER BY order_date DESC

) rn

FROM orders

)
```

```
SELECT *

FROM ranked_data

WHERE rn=1;
```

What Interviewers Check

- CDC Logic
 - Deduplication
 - Production ETL Scenarios
-

Question 10

Calculate customer session duration.

Answer

```
SELECT

l.customer_id,

TIMESTAMPDIFF(

MINUTE,

l.event_time,

lo.event_time

) session_minutes
```

```
FROM customer_events l  
JOIN customer_events lo  
ON l.customer_id=lo.customer_id  
WHERE l.event_type='login'  
AND lo.event_type='logout';
```

What Interviewers Check

- Event Processing
 - Time Calculations
 - Session Analytics
-

Frequently Asked Theoretical SQL Questions

What is the difference between ROW_NUMBER, RANK and DENSE_RANK?

ROW_NUMBER assigns unique numbers.

RANK skips numbers when ties occur.

DENSE_RANK does not skip numbers when ties occur.

Why is NOT EXISTS preferred over NOT IN?

NOT EXISTS handles NULL values safely.

NOT IN can return unexpected results when NULL values are present.

In production systems, NOT EXISTS is generally preferred.

When would you use CTE instead of Subquery?

Use CTEs when:

- Query readability matters
- Logic needs to be reused
- Multiple transformations are involved

CTEs make debugging easier.

What is the difference between WHERE and HAVING?

WHERE filters rows before aggregation.

HAVING filters aggregated results after GROUP BY.

What causes duplicate records after JOIN?

Common reasons:

- Missing join conditions
 - One-to-many relationships
 - Many-to-many relationships
 - Data quality issues
-

Difference Between DELETE, TRUNCATE and DROP

DELETE:

- Removes selected rows
- Can use WHERE clause
- Logged row by row

TRUNCATE:

- Removes all rows
- Faster than DELETE
- Cannot use WHERE

DROP:

- Removes complete table structure
-

Explain Window Functions

Window functions perform calculations across a set of rows while preserving row-level details.

Common examples:

ROW_NUMBER()

RANK()

DENSE_RANK()

LAG()

LEAD()

SUM() OVER()

AVG() OVER()

Window functions are heavily used in Data Engineering pipelines.

How Do You Handle Duplicate Data in ETL Pipelines?

Common approaches:

- ROW_NUMBER based deduplication
- Merge operations
- Primary key validations
- Hash key comparisons
- CDC implementations

What SQL Topics Are Most Frequently Asked in Data Engineering Interviews?

1. Joins
2. Window Functions
3. CTEs
4. Aggregations
5. Subqueries
6. Data Deduplication
7. Date Functions
8. Event-Based Problems
9. Optimization Concepts
10. Data Warehousing Scenarios

Final Advice

Do not prepare SQL by memorizing syntax.

Most Data Engineering interviews evaluate:

- Problem-solving ability
- Data transformation thinking
- Business logic understanding
- Query optimization mindset
- Real-world application of SQL

Focus on understanding why a query is written, not just how it is written.